Express Routing — Route Definitions, Parameters & the Express Router

Section: 7 — Express.js Framework Basics

Subtopic: 02 — Routing

Estimated Duration: 3–4 hours

Theory Practical

Topics covered: Route definition with `app.get()` / `post()` / `put()` / `delete()`, route parameters with `:id` and `req.params`, query strings vs parameters, route ordering and specificity, `app.all()` and `app.route()`, Express Router for modular routing, mounting routers with `app.use()`

Learning Objectives — This Subtopic

- Define routes for all standard HTTP methods (GET, POST, PUT, DELETE) and explain when each is used
- Extract dynamic values from URLs using route parameters (`:param`) and access them via `req.params`
- Distinguish between route parameters, query strings, and request bodies as data sources
- Use `app.route()` to chain multiple method handlers on a single path
- Create modular route files using `express.Router()` and mount them with `app.use()`
- Organise a multi-resource application with a clean, scalable folder structure

1. HTTP Methods and Route Definition

In subtopic 07-01, you used `app.get()` to handle GET requests. Express provides a matching method for every standard HTTP verb. Each method maps to a specific operation in the context of a REST API:

Express Method	HTTP Verb	Typical Purpose
`app.get(path, handler)`	GET	Retrieve a resource or list of resources
`app.post(path, handler)`	POST	Create a new resource
`app.put(path, handler)`	PUT	Replace an entire resource
`app.patch(path, handler)`	PATCH	Partially update a resource
`app.delete(path, handler)`	DELETE	Remove a resource

Here is a minimal example demonstrating all five methods on a single resource:

http-methods.js

```
const express = require('express');
const app = express();

// Required for parsing JSON request bodies (covered fully in Section 8)
app.use(express.json());

app.get('/products', (req, res) => {
  res.json({ action: 'GET — list all products' });
});

app.post('/products', (req, res) => {
  res.status(201).json({ action: 'POST — create a product', received: req.body });
});

app.put('/products/1', (req, res) => {
  res.json({ action: 'PUT — replace product 1', received: req.body });
});

app.patch('/products/1', (req, res) => {
  res.json({ action: 'PATCH — update product 1 partially', received: req.body });
});

app.delete('/products/1', (req, res) => {
  res.json({ action: 'DELETE — remove product 1' });
});
```

```
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Run it:

```
node http-methods.js
```

> **Output:**
>
> ```
> Server is running on http://localhost:3000
> ```

Test each method with `curl`:

```
curl http://localhost:3000/products
```

> **Output:**
>
> ```
> {"action":"GET — list all products"}
> ```

```
curl -X POST http://localhost:3000/products -H "Content-Type: application/json" -d '{"name":"Keyboard"}'
```

> **Output:**
>
> ```
> {"action":"POST — create a product","received":{"name":"Keyboard"}}
> ```

```
curl -X DELETE http://localhost:3000/products/1
```

> **Output:**
>
> ```
> {"action":"DELETE — remove product 1"}
> ```

What is happening:

- The same URL path (`/products`) can have different handlers for different HTTP methods. The combination of **method + path** uniquely identifies a route.

- `app.use(express.json())` is a built-in middleware that parses incoming JSON request bodies and populates `req.body`. Without it, `req.body` is `undefined`. Middleware is covered in detail in subtopic 07-03, but you need this line to test POST and PUT routes.
- `res.status(201)` sets a "201 Created" status code, which is the conventional response for successful resource creation.

> **Note:** Browsers can only send GET requests via the address bar. To test POST, PUT, PATCH, and DELETE routes, use `curl` from the terminal or a tool like Postman. The `-X` flag in curl specifies the HTTP method.

---

2. Route Parameters

Hard-coding IDs into route paths (`/products/1`) is obviously impractical. Route parameters let you define **dynamic segments** in the URL path. A parameter is prefixed with a colon (`:`) in the route definition and its value is captured in `req.params`.

Stage 1 A Single Parameter

params-basic.js

```
const express = require('express');
const app = express();

const users = [
  { id: 1, name: 'Alice', role: 'admin' },
  { id: 2, name: 'Bob', role: 'editor' },
  { id: 3, name: 'Charlie', role: 'viewer' }
];

// List all users
app.get('/users', (req, res) => {
  res.json(users);
});

// Get a single user by ID
app.get('/users/:id', (req, res) => {
```

```
  const userId = parseInt(req.params.id, 10);
  const user = users.find(u => u.id === userId);

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }

  res.json(user);
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node params-basic.js`

Output:

```
Server is running on http://localhost:3000
```

`curl http://localhost:3000/users/2`

Output:

```
{"id":2,"name":"Bob","role":"editor"}
```

`curl http://localhost:3000/users/99`

Output:

```
{"error":"User not found"}
```

What is happening:

- `:id` in the route path `/users/:id` defines a named parameter. When a request arrives at `/users/2`, Express captures `"2"` and stores it in `req.params.id`.
- **Route parameter values are always strings.** The URL `/users/2` gives `req.params.id === "2"` (a string), not `2` (a number). You must parse it with `parseInt()` or `Number()` before comparing with numeric IDs.

- The `return` before `res.status(404).json(...)` is critical. Without it, execution continues past the `if` block and Express attempts to send a second response, which throws an error.

> **Warning:** A common mistake is forgetting that `req.params` values are strings. This comparison silently fails:
> `users.find(u => u.id === req.params.id)`
> Because `1 === "1"` is `false` in JavaScript (strict equality). Always parse to a number first, or use `==` (loose equality) — though parsing is the more explicit and recommended approach.

Stage 2 Multiple Parameters

Routes can have more than one parameter. Each colon-prefixed segment becomes a separate key in `req.params`:

params-multiple.js

```
const express = require('express');
const app = express();

const posts = [
  { userId: 1, postId: 101, title: 'First Post' },
  { userId: 1, postId: 102, title: 'Second Post' },
  { userId: 2, postId: 201, title: 'Bob Writes' }
];

// Route with two parameters
app.get('/users/:userId/posts/:postId', (req, res) => {
  const userId = parseInt(req.params.userId, 10);
  const postId = parseInt(req.params.postId, 10);

  const post = posts.find(p => p.userId === userId && p.postId === postId);

  if (!post) {
    return res.status(404).json({ error: 'Post not found' });
  }

  res.json(post);
});

// List all posts for a user
app.get('/users/:userId/posts', (req, res) => {
```

```
  const userId = parseInt(req.params.userId, 10);
  const userPosts = posts.filter(p => p.userId === userId);
  res.json(userPosts);
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node params-multiple.js`

Output:

```
Server is running on http://localhost:3000
```

`curl http://localhost:3000/users/1/posts`

Output:

```
[{"userId":1,"postId":101,"title":"First Post"},{"userId":1,"postId":102,"title":"Second Post"}]
```

`curl http://localhost:3000/users/1/posts/102`

Output:

```
{"userId":1,"postId":102,"title":"Second Post"}
```

The URL pattern `/users/:userId/posts/:postId` represents a **nested resource** — a post that belongs to a specific user. This is a common REST API convention. The `req.params` object for the URL `/users/1/posts/102` is:

```
{ userId: '1', postId: '102' }
```

3. Route Parameters vs Query Strings

There are two ways to pass data through a URL, and they serve different purposes:

Mechanism	URL Example	Access	Use Case
Route parameters	`/users/42`	`req.params.id`	Identify a **specific resource** (required)
Query strings	`/users?role=admin`	`req.query.role`	Filter, sort, or paginate (optional)

Here is a practical example combining both:

params-vs-query.js

```
const express = require('express');
const app = express();

const products = [
  { id: 1, name: 'Laptop', category: 'electronics', price: 999 },
  { id: 2, name: 'Headphones', category: 'electronics', price: 149 },
  { id: 3, name: 'Desk Chair', category: 'furniture', price: 349 },
  { id: 4, name: 'Monitor', category: 'electronics', price: 499 },
  { id: 5, name: 'Bookshelf', category: 'furniture', price: 199 }
];

// GET /products?category=electronics&sort=price
// Query strings for filtering and sorting
app.get('/products', (req, res) => {
  let result = [...products];

  // Filter by category (optional)
  if (req.query.category) {
    result = result.filter(p => p.category === req.query.category);
  }

  // Sort by field (optional)
  if (req.query.sort) {
    const field = req.query.sort;
    result.sort((a, b) => (a[field] > b[field] ? 1 : -1));
  }
```

```
  res.json(result);
});

// GET /products/3
// Route parameter for a specific resource
app.get('/products/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  const product = products.find(p => p.id === id);

  if (!product) {
    return res.status(404).json({ error: 'Product not found' });
  }

  res.json(product);
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node params-vs-query.js
```

Output:

```
Server is running on http://localhost:3000
```

```
curl "http://localhost:3000/products?category=electronics&sort=price"
```

Output:

```
[{"id":2,"name":"Headphones","category":"electronics","price":149},{"id":
4,"name":"Monitor","category":"electronics","price":499},{"id":
1,"name":"Laptop","category":"electronics","price":999}]
```

```
curl http://localhost:3000/products/3
```

Output:

```
{"id":3,"name":"Desk Chair","category":"furniture","price":349}
```

The guideline: Use route parameters when the value *identifies* the resource (you cannot return a meaningful response without it). Use query strings when the value *modifies* the response (filtering, sorting, pagination) and the request still works without it.

> **Analogy:** Think of route parameters as the address of a house (`/street/42` — you need it to find the right place). Query strings are delivery instructions (`?leave_at=back_door` — helpful but optional; the parcel still arrives at the right house without them).

---

4. Route Ordering and Specificity

Express matches routes in the order they are **registered**. The first route whose pattern matches the incoming request wins. This means you must define more specific routes *before* more general ones.

4.1 The Classic Ordering Problem

`route-order-wrong.js`

```
const express = require('express');
const app = express();

// WRONG ORDER — the parameter route catches everything
app.get('/users/:id', (req, res) => {
  res.json({ route: 'parameter', id: req.params.id });
});

// This route will NEVER be reached
app.get('/users/active', (req, res) => {
  res.json({ route: 'active users' });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node route-order-wrong.js
```

Output:

```
Server is running on http://localhost:3000
```

```
curl http://localhost:3000/users/active
```

Output:

```
{"route":"parameter","id":"active"}
```

The request to `/users/active` matched `/users/:id` first, with `req.params.id` set to the string `"active"`. The intended `/users/active` handler never ran.

4.2 The Fix: Specific Routes First

`route-order-correct.js`

```
const express = require('express');
const app = express();

// CORRECT ORDER — specific route first
app.get('/users/active', (req, res) => {
  res.json({ route: 'active users' });
});

app.get('/users/:id', (req, res) => {
  res.json({ route: 'parameter', id: req.params.id });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node route-order-correct.js
```

Output:

```
Server is running on http://localhost:3000
```

```
curl http://localhost:3000/users/active
```

Output:

```
{"route":"active users"}
```

```
curl http://localhost:3000/users/5
```

Output:

```
{"route":"parameter","id":"5"}
```

Now `/users/active` matches the literal route, and `/users/5` falls through to the parameter route. The rule is straightforward: **static segments before dynamic parameters, specific patterns before general ones.**

> **Important:** This ordering requirement applies within the same HTTP method. A `POST /users/:id` route does not conflict with a `GET /users/active` route because they respond to different HTTP verbs.

---

5. `app.all()` and `app.route()`

5.1 `app.all()` — Match Any HTTP Method

`app.all(path, handler)` registers a handler that runs for *any* HTTP method on the given path. This is useful for tasks that apply regardless of the method, such as logging or access control:

app-all.js

```
const express = require('express');
const app = express();

app.use(express.json());
```

```
// Runs for ANY method on /api/secret
app.all('/api/secret', (req, res) => {
  res.status(403).json({
    error: 'Access denied',
    method: req.method,
    message: 'This endpoint is not available'
  });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node app-all.js`

> Output:
>
> ```
> Server is running on http://localhost:3000
> ```

`curl http://localhost:3000/api/secret`

> Output:
>
> ```
> {"error":"Access denied","method":"GET","message":"This endpoint is not available"}
> ```

`curl -X POST http://localhost:3000/api/secret`

> Output:
>
> ```
> {"error":"Access denied","method":"POST","message":"This endpoint is not available"}
> ```

5.2 `app.route()` — Chaining Methods on a Path

When multiple HTTP methods share the same path, `app.route()` lets you chain them without repeating the path string. This reduces duplication and groups related handlers together:

`app-route.js`

```
const express = require('express');
const app = express();

app.use(express.json());

const tasks = [
  { id: 1, title: 'Learn Express routing', done: false },
  { id: 2, title: 'Build a REST API', done: false }
];

app.route('/tasks')
  .get((req, res) => {
    res.json(tasks);
  })
  .post((req, res) => {
    const newTask = {
      id: tasks.length + 1,
      title: req.body.title,
      done: false
    };
    tasks.push(newTask);
    res.status(201).json(newTask);
  });

app.route('/tasks/:id')
  .get((req, res) => {
    const task = tasks.find(t => t.id === parseInt(req.params.id, 10));
    if (!task) return res.status(404).json({ error: 'Task not found' });
    res.json(task);
  })
  .put((req, res) => {
    const task = tasks.find(t => t.id === parseInt(req.params.id, 10));
    if (!task) return res.status(404).json({ error: 'Task not found' });
    task.title = req.body.title || task.title;
    task.done = req.body.done !== undefined ? req.body.done : task.done;
    res.json(task);
  })
  .delete((req, res) => {
    const index = tasks.findIndex(t => t.id === parseInt(req.params.id, 10));
    if (index === -1) return res.status(404).json({ error: 'Task not found' });
    const removed = tasks.splice(index, 1);
    res.json(removed[0]);
  });
```

```
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node app-route.js`

Output:

```
Server is running on http://localhost:3000
```

`curl http://localhost:3000/tasks`

Output:

```
[{"id":1,"title":"Learn Express routing","done":false},{"id":2,"title":"Build a REST API","done":false}]
```

`curl -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Write tests"}'`

Output:

```
{"id":3,"title":"Write tests","done":false}
```

`curl -X PUT http://localhost:3000/tasks/1 -H "Content-Type: application/json" -d '{"done":true}'`

Output:

```
{"id":1,"title":"Learn Express routing","done":true}
```

`curl -X DELETE http://localhost:3000/tasks/2`

Output:

```
{"id":2,"title":"Build a REST API","done":false}
```

What is happening: `app.route('/tasks')` returns a chainable route object. Each chained method (`.get()`, `.post()`, etc.) registers a handler for that HTTP verb on the same path. This is functionally identical to writing separate `app.get('/tasks', ...)` and `app.post('/tasks', ...)` calls, but more concise and easier to maintain.

> **Key Insight:** The tasks example above is a complete, working CRUD API (Create, Read, Update, Delete). The data lives in an in-memory array, which means it resets every time the server restarts. In Section 10–12, you will replace this with a TypeORM-backed database for persistent storage.

---

6. Express Router — Modular Routing

Defining every route in a single file works for small applications, but a real API with dozens of endpoints becomes unmanageable quickly. `express.Router()` creates a mini-application (a "router") that you can define routes on, then mount onto the main app at a specific path prefix.

> **Analogy:** Think of the main Express app as a building's reception desk. Each `Router` is a department on a specific floor. The reception desk (main app) directs visitors to the right floor (router), and the department handles them from there. The department does not need to know about other floors.

Stage 3 Extracting Routes into Separate Files

Start by creating a project structure:

```
express-router-demo/
├── routes/
│   ├── users.js
│   └── products.js
```

```
├── app.js
├── package.json
└── package-lock.json
```

Initialise the project and install Express:

```
mkdir express-router-demo && cd express-router-demo
```

```
npm init -y
```

```
npm install express
```

```
mkdir routes
```

Step 1: Create the Users Router

routes/users.js

```
const express = require('express');
const router = express.Router();

const users = [
  { id: 1, name: 'Alice', email: 'alice@example.com' },
  { id: 2, name: 'Bob', email: 'bob@example.com' },
  { id: 3, name: 'Charlie', email: 'charlie@example.com' }
];

// GET /users
router.get('/', (req, res) => {
  res.json(users);
});

// GET /users/:id
router.get('/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id, 10));
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});

// POST /users
router.post('/', (req, res) => {
  const newUser = {
    id: users.length + 1,
```

```
    name: req.body.name,
    email: req.body.email
  };
  users.push(newUser);
  res.status(201).json(newUser);
});

// DELETE /users/:id
router.delete('/:id', (req, res) => {
  const index = users.findIndex(u => u.id === parseInt(req.params.id, 10));
  if (index === -1) return res.status(404).json({ error: 'User not found' });
  const removed = users.splice(index, 1);
  res.json(removed[0]);
});

module.exports = router;
```

Step 2: Create the Products Router

routes/products.js

```
const express = require('express');
const router = express.Router();

const products = [
  { id: 1, name: 'Laptop', price: 999 },
  { id: 2, name: 'Headphones', price: 149 },
  { id: 3, name: 'Monitor', price: 499 }
];

// GET /products
router.get('/', (req, res) => {
  // Support optional price filtering via query string
  let result = [...products];

  if (req.query.maxPrice) {
    const max = parseFloat(req.query.maxPrice);
    result = result.filter(p => p.price <= max);
  }

  res.json(result);
});

// GET /products/:id
router.get('/:id', (req, res) => {
```

```
  const product = products.find(p => p.id === parseInt(req.params.id, 10));
  if (!product) return res.status(404).json({ error: 'Product not found' });
  res.json(product);
});

module.exports = router;
```

Step 3: Mount the Routers in the Main App

app.js

```
const express = require('express');
const app = express();

const usersRouter = require('./routes/users');
const productsRouter = require('./routes/products');

// Parse JSON bodies
app.use(express.json());

// Mount routers with path prefixes
app.use('/users', usersRouter);
app.use('/products', productsRouter);

// Root route
app.get('/', (req, res) => {
  res.json({
    message: 'API is running',
    endpoints: ['/users', '/products']
  });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

```
node app.js
```

> **Output:**
>
> ```
> Server is running on http://localhost:3000
> ```

```
curl http://localhost:3000/users
```

Output:

```
[{"id":1,"name":"Alice","email":"alice@example.com"},{"id":2,"name":"Bob","email":"bob@example.com"},{"id":3,"name":"Charlie","email":"charlie@example.com"}]
```

```
curl http://localhost:3000/products?maxPrice=200
```

Output:

```
[{"id":2,"name":"Headphones","price":149}]
```

```
curl -X POST http://localhost:3000/users -H "Content-Type: application/json" -d '{"name":"Diana","email":"diana@example.com"}'
```

Output:

```
{"id":4,"name":"Diana","email":"diana@example.com"}
```

What is happening:

- `express.Router()` creates a mini-app with its own `.get()`, `.post()`, `.put()`, `.delete()` methods — identical to the main `app`.
- Inside `routes/users.js`, routes are defined relative to the router's own root (`/`, `/:id`). The router has no knowledge of its eventual mount point.
- `app.use('/users', usersRouter)` mounts the router at `/users`. The router's `/` becomes `/users`, and its `/:id` becomes `/users/:id`.
- Each router file exports its router via `module.exports = router`, and the main file imports it with `require()`.

> **Key Insight:** Routers can be nested. A `usersRouter` could mount a `postsRouter` at `/:userId/posts`, creating deeply nested resource paths like `/users/1/posts/42`. The same modular pattern scales to any depth.

7. Router-Level Organisation Patterns

7.1 Index File for Routes

As the number of routers grows, importing each one individually in `app.js` becomes cumbersome. A common pattern is to create a `routes/index.js` file that aggregates all routers:

```
express-router-demo/
├── routes/
│   ├── index.js       <-- new: aggregates all routers
│   ├── users.js
│   └── products.js
├── app.js
├── package.json
└── package-lock.json
```

routes/index.js

```
const express = require('express');
const router = express.Router();

const usersRouter = require('./users');
const productsRouter = require('./products');

router.use('/users', usersRouter);
router.use('/products', productsRouter);

module.exports = router;
```

Now `app.js` becomes cleaner:

app.js (updated)

```
const express = require('express');
const app = express();

const routes = require('./routes');
```

```
app.use(express.json());

// Mount all routes under /api
app.use('/api', routes);

app.get('/', (req, res) => {
  res.json({ message: 'API is running', base: '/api' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

`node app.js`

Output:

```
Server is running on http://localhost:3000
```

`curl http://localhost:3000/api/users`

Output:

```
[{"id":1,"name":"Alice","email":"alice@example.com"},{"id":2,"name":"Bob","email":"bob@example.com"},{"id":3,"name":"Charlie","email":"charlie@example.com"}]
```

`curl http://localhost:3000/api/products/1`

Output:

```
{"id":1,"name":"Laptop","price":999}
```

What is happening: The `routes/index.js` file itself is a router that mounts child routers. The main app mounts this aggregate router at `/api`. The final URL paths are `/api/users`, `/api/users/:id`, `/api/products`, etc. This three-level composition (app → index router → resource routers) is the standard pattern in production Express applications.

7.2 Using `app.route()` Inside Routers

The `route()` method works on routers too. Combine it with the Router for maximum conciseness:

routes/users.js (refactored excerpt)

```
const express = require('express');
const router = express.Router();

// ... users array ...

router.route('/')
  .get((req, res) => {
    res.json(users);
  })
  .post((req, res) => {
    // create logic
  });

router.route('/:id')
  .get((req, res) => {
    // find by id
  })
  .put((req, res) => {
    // update logic
  })
  .delete((req, res) => {
    // delete logic
  });

module.exports = router;
```

This is a configuration file showing a pattern, so no output box is needed. The key takeaway is that `router.route()` works identically to `app.route()`.

---

8. Common Patterns and Gotchas

8.1 Trailing Slashes

By default, Express treats `/users` and `/users/` as the same route. However, this can be changed with the `strict routing` setting:

strict-routing.js

```
const express = require('express');
const app = express();

// Enable strict routing — /users and /users/ are now DIFFERENT
app.set('strict routing', true);

app.get('/users', (req, res) => {
  res.send('Without trailing slash');
});

app.get('/users/', (req, res) => {
  res.send('With trailing slash');
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node strict-routing.js
```

> Output:
>
> ```
> Server is running on http://localhost:3000
> ```

```
curl http://localhost:3000/users
```

> Output:
>
> ```
> Without trailing slash
> ```

```
curl http://localhost:3000/users/
```

Output:

```
With trailing slash
```

In most applications, the default behaviour (treating both as equivalent) is preferred. Enabling strict routing is rare but worth knowing about.

8.2 Case Sensitivity

By default, Express routes are **case-insensitive**: `/Users`, `/users`, and `/USERS` all match a route defined as `/users`. To change this:

`case-sensitive.js`

```
const express = require('express');
const app = express();

app.set('case sensitive routing', true);

app.get('/users', (req, res) => {
  res.send('Lowercase users');
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node case-sensitive.js
```

Output:

```
Server is running on http://localhost:3000
```

```
curl http://localhost:3000/users
```

Output:

```
Lowercase users
```

```
curl http://localhost:3000/Users
```

> Output:
>
> ```
> Cannot GET /Users
> ```

8.3 Sending a Response Exactly Once

Every route handler must send exactly one response. Sending two responses (or none) causes errors:

double-response.js

```
const express = require('express');
const app = express();

// BUG: sends two responses
app.get('/bad', (req, res) => {
  res.json({ message: 'First response' });
  res.json({ message: 'Second response' });  // Error!
});

// CORRECT: use return to exit after sending
app.get('/good', (req, res) => {
  const id = parseInt(req.query.id, 10);

  if (isNaN(id)) {
    return res.status(400).json({ error: 'Invalid ID' });
  }

  res.json({ message: 'Valid request', id: id });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node double-response.js
```

> Output:
>
> ```
> Server is running on http://localhost:3000
> ```

```
curl http://localhost:3000/bad
```

Output:

```
{"message":"First response"}
```

The client receives the first response, but the server logs an error in the terminal:

Output:

```
Error [ERR_HTTP_HEADERS_SENT]: Cannot set headers after they are sent to the client
```

```
curl "http://localhost:3000/good?id=abc"
```

Output:

```
{"error":"Invalid ID"}
```

```
curl "http://localhost:3000/good?id=42"
```

Output:

```
{"message":"Valid request","id":42}
```

Warning: The “Cannot set headers after they are sent” error is one of the most common Express bugs. It occurs when a handler attempts to send a response after one has already been sent. The fix is almost always adding `return` before early responses (like error checks) to prevent the function from continuing.

Summary

Concept	Key Point
HTTP method routes	`app.get()`, `.post()`, `.put()`, `.patch()`, `.delete(` — one per verb
Route parameters	`:param` in path → `req.params.param` (always a string)
Params vs query	Parameters identify resources (required); query strings filter/modify (optional)
Route ordering	Specific/static routes before dynamic/ parameterised routes
`app.all()`	Matches any HTTP method on a given path
`app.route()`	Chain multiple method handlers on one path — reduces repetition
`express.Router()`	Creates a modular mini-app; define routes relative to router root
Mounting routers	`app.use('/prefix', router)` — router's `/` becomes `/prefix`
Index router pattern	Aggregate routers in `routes/index.js` for clean imports
Double-response bug	Always `return` after early responses to prevent "headers already sent" errors

Interview Questions

1. What is the difference between `req.params` and `req.query`?
Expected answer: `req.params` contains values extracted from named route segments (e.g., `/users/:id` → `req.params.id`). `req.query` contains key-value pairs parsed from the URL query string (e.g., `?page=2` → `req.query.page`). Parameters are part of the route definition and typically required; query strings are optional modifiers.

2. Why does route definition order matter in Express?
Expected answer: Express matches routes in registration order and invokes the first matching handler. If a parameterised route like `/users/:id` is registered before a static route like `/users/active`, the parameter route captures `"active"` as the `:id` value and the static route never executes. Specific routes must be registered before general ones.

3. What is `express.Router()` and why would you use it?
Expected answer: `express.Router()` creates an isolated, modular set of routes (a "mini-app"). You define routes on the router, export it from a file, and mount it in the main app with `app.use(prefix, router)`. It improves code organisation, testability, and separation of concerns as the application scales.

4. Explain the difference between `app.get('/path', handler)` and `app.all('/path', handler)`.
Expected answer: `app.get()` only matches GET requests to the path. `app.all()` matches *any* HTTP method (GET, POST, PUT, DELETE, etc.) to the path. `app.all()` is useful for cross-cutting concerns like logging or blanket access control that should apply regardless of the HTTP method.

5. What type are route parameter values, and why does this matter?
Expected answer: Route parameter values are always **strings**. If your data uses numeric IDs, a strict equality check like `user.id === req.params.id` fails because `1 === "1"` is `false`. You must parse the parameter to a number (e.g., `parseInt(req.params.id, 10)`) before comparison.

6. What causes the "Cannot set headers after they are sent to the client" error?
Expected answer: This occurs when a route handler attempts to send a second response after one has already been sent. The typical cause is a missing `return` statement before an early response (such as an error check), which allows execution to continue to a second `res.send()` or `res.json()` call.

7. How does mounting a router with `app.use('/api', router)` affect the paths defined inside that router?
Expected answer: The mount path (`/api`) is prepended to every route defined in the router. If the router defines `router.get('/users', ...)` , the effective URL becomes `/api/users` . Inside the router, routes are defined relative to the mount point — the router does not know or care what prefix it is mounted at.

8. What does `app.route('/tasks')` return, and how is it different from `app.get('/tasks', ...)` ?
Expected answer: `app.route('/tasks')` returns a chainable route object for the given path. You can call `.get()` , `.post()` , `.put()` , etc. on it without repeating the path. It is a convenience that groups all method handlers for one path together, improving readability. Functionally, the resulting routes are identical to individually registered ones.

Practical Assignment: Modular Bookshop API

Objective: Build a RESTful API for a bookshop with modular routing, route parameters, query string filtering, and full CRUD operations.

Instructions:

1. Create a new project called `bookshop-api` . Initialise it and install Express.
2. Create the following folder structure:

```
bookshop-api/
├── routes/
│   ├── index.js
│   ├── books.js
│   └── authors.js
├── app.js
└── package.json
```

3. In `routes/books.js`, define an in-memory array of books (at least 5) with `id`, `title`, `authorId`, `genre`, and `year` fields. Implement these routes on a router:
   - `GET /` — List all books. Support optional query parameters `?genre=fiction` and `?year=2020` for filtering.
   - `GET /:id` — Get a single book by ID. Return 404 if not found.
   - `POST /` — Create a new book from the request body. Return 201.
   - `PUT /:id` — Replace a book's data. Return 404 if not found.
   - `DELETE /:id` — Remove a book. Return 404 if not found.
4. In `routes/authors.js`, define an in-memory array of authors (at least 3) with `id` and `name` fields. Implement:
   - `GET /` — List all authors.
   - `GET /:id` — Get a single author by ID.
5. In `routes/index.js`, aggregate both routers and export the combined router.
6. In `app.js`, mount the combined router at `/api`. Add a `GET /` root route that returns a JSON welcome message listing the available endpoints.
7. Test every endpoint using `curl`.

Expected Output:

```
curl http://localhost:3000/api/books?genre=fiction
```

Output:

```
[{"id":1,"title":"The Great Gatsby","authorId":1,"genre":"fiction","year":1925},{"id":
3,"title":"1984","authorId":3,"genre":"fiction","year":1949}]
```

```
curl http://localhost:3000/api/books/2
```

Output:

```
{"id":2,"title":"A Brief History of Time","authorId":2,"genre":"science","year":1988}
```

```
curl -X POST http://localhost:3000/api/books -H "Content-Type: application/json" -d '{"title":"Dune","authorId":4,"genre":"fiction","year":1965}'
```

Output:

```
{"id":6,"title":"Dune","authorId":4,"genre":"fiction","year":1965}
```

```
curl http://localhost:3000/api/authors/1
```

Output:

```
{"id":1,"name":"F. Scott Fitzgerald"}
```

Bonus Challenge: Add a nested route `GET /authors/:authorId/books` that returns all books by a specific author. Implement this by having the authors router reference the books data (hint: extract the books array into a shared module, or mount a sub-router).